

Netflix *Movies* Recommendation Systems

Bartosz Maj

Creating the dataset



Input Data Structure

```
MovieID: 1:  
UserID,Rating,Date  
2343,4,2005-03-12  
1234,5,2005-03-14
```



Data Processing Pipeline

process_netflix_file()

- Parses raw data files
- Converts to structured DataFrame
- Handles date formatting



Dataset Creation

create_netflix_dataset()

- Combines multiple data files
- Filters by minimum ratings
- Creates train/test split



Quality Assurance

Input Validation

- Date format checking
- Rating range validation

Data Integrity

- Stratified sampling
- Complete coverage check

Filtering users and movies on minimal ratings

```
if min_ratings > 0:
    # Filter users and movies with too few ratings
    user_counts = full_training_df['user_id'].value_counts()
    movie_counts = full_training_df['movie_id'].value_counts()

    valid_users = user_counts[user_counts >= min_ratings].index
    valid_movies = movie_counts[movie_counts >= min_ratings].index

    # Filter training data
    full_training_df = full_training_df[
        full_training_df['user_id'].isin(valid_users) &
        full_training_df['movie_id'].isin(valid_movies)
    ]

    # Create train/test split
    print("Creating train/test split...")
    train_df, test_df = stratified_rating_split(
        full_training_df,
        test_size=test_size,
        random_state=random_state
    )

    return train_df, test_df
```

Verification that all users and movies are in train and test datasets

```
# Create stratification based on rating values
train_idx, test_idx = train_test_split(
    np.arange(len(df)),
    test_size=test_size,
    random_state=random_state,
    stratify=df['rating'] # Maintain rating distribution
)

train_df = df.iloc[train_idx].copy()
test_df = df.iloc[test_idx].copy()

# Verify all users and movies appear in both sets
train_users = set(train_df['user_id'].unique())
train_movies = set(train_df['movie_id'].unique())
test_users = set(test_df['user_id'].unique())
test_movies = set(test_df['movie_id'].unique())

# Move some test ratings to train if necessary
missing_user_mask = ~test_df['user_id'].isin(train_users)
missing_movie_mask = ~test_df['movie_id'].isin(train_movies)

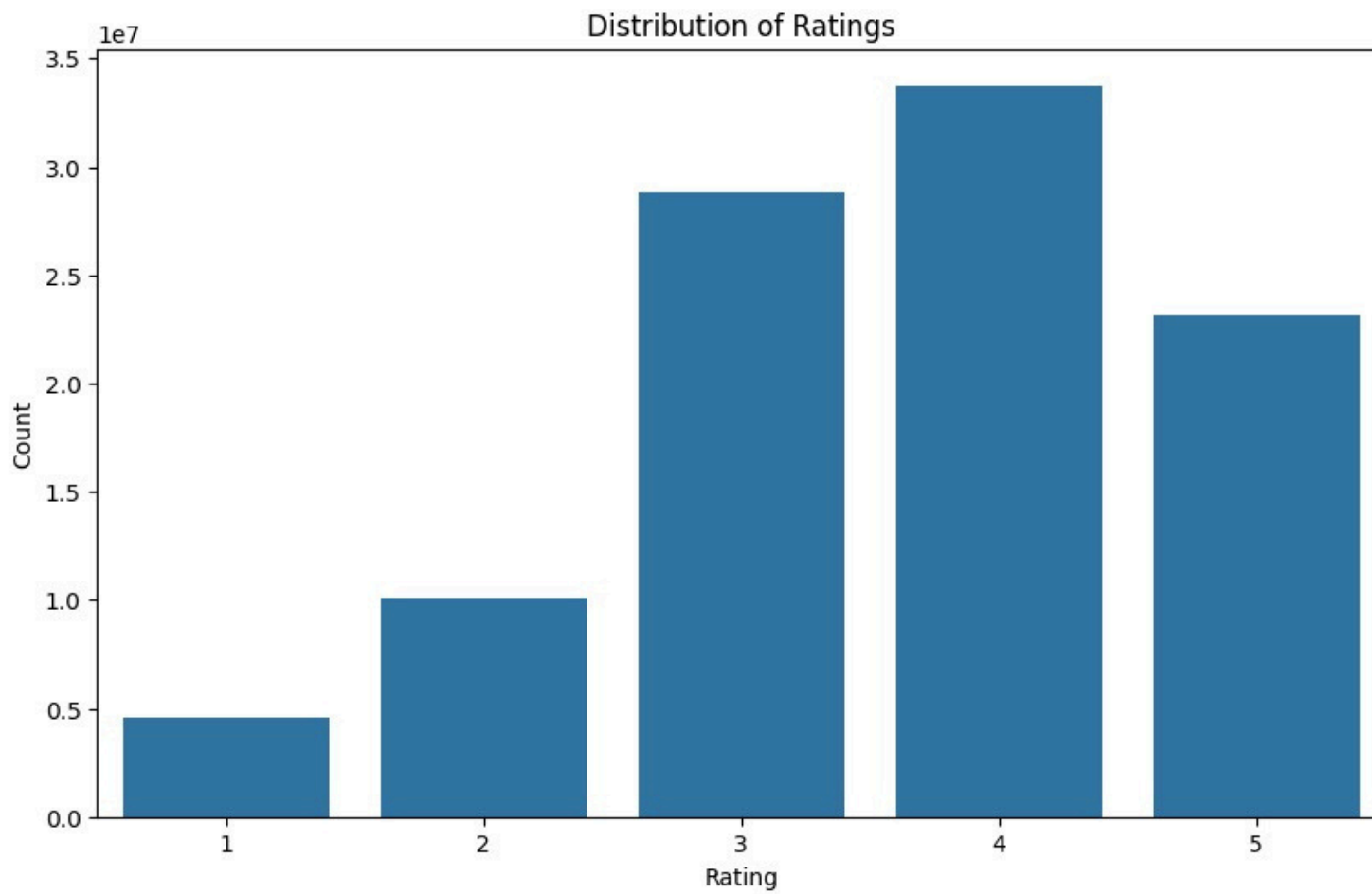
if missing_user_mask.any() or missing_movie_mask.any():
    move_to_train = test_df[missing_user_mask | missing_movie_mask]
    test_df = test_df[~(missing_user_mask | missing_movie_mask)]
    train_df = pd.concat([train_df, move_to_train], ignore_index=True)

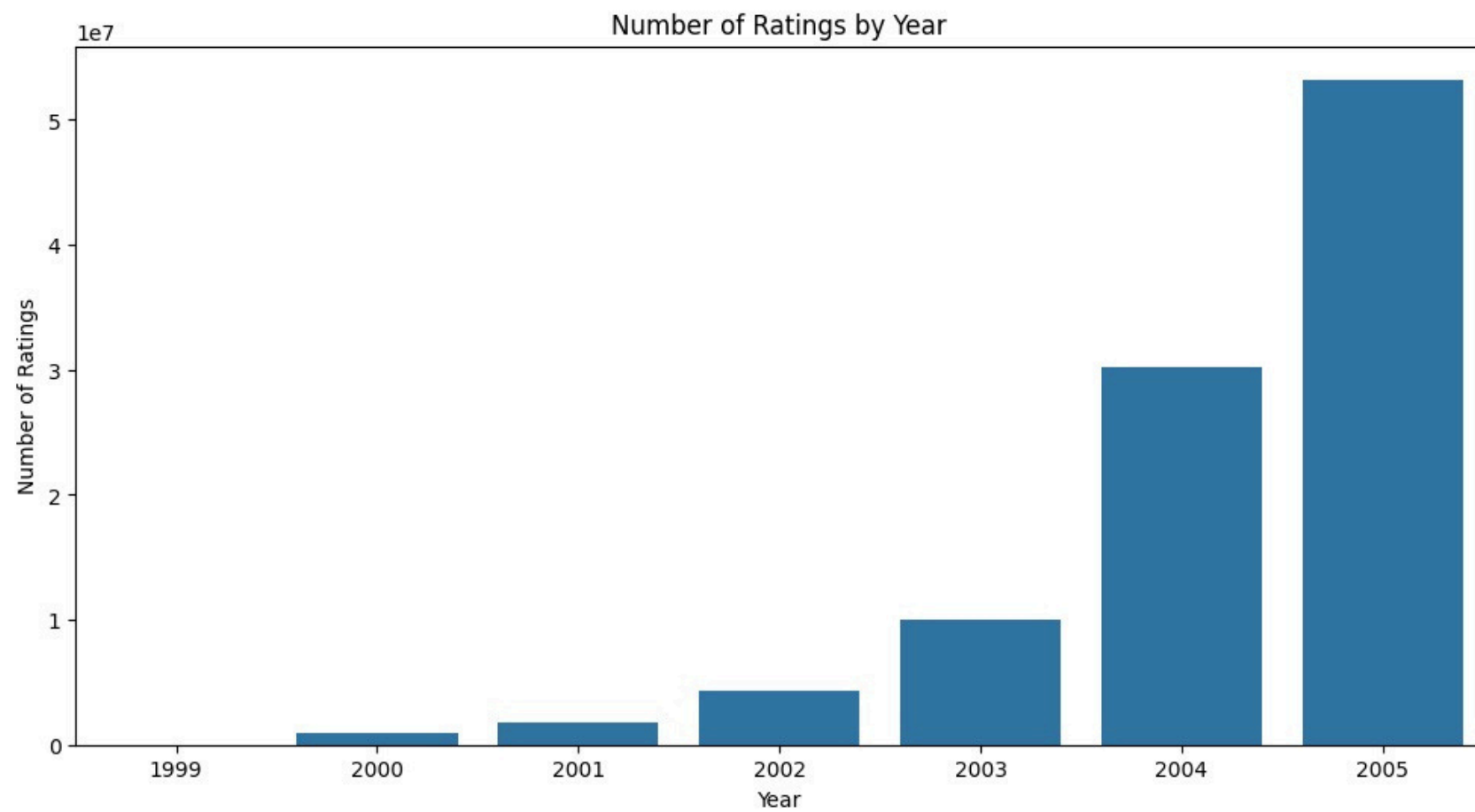
print(f"Train set size: {len(train_df)}, Test set size: {len(test_df)}")
print(f"Train ratings distribution:\n{train_df['rating'].value_counts(normalize=True)}")
print(f"Test ratings distribution:\n{test_df['rating'].value_counts(normalize=True)}")

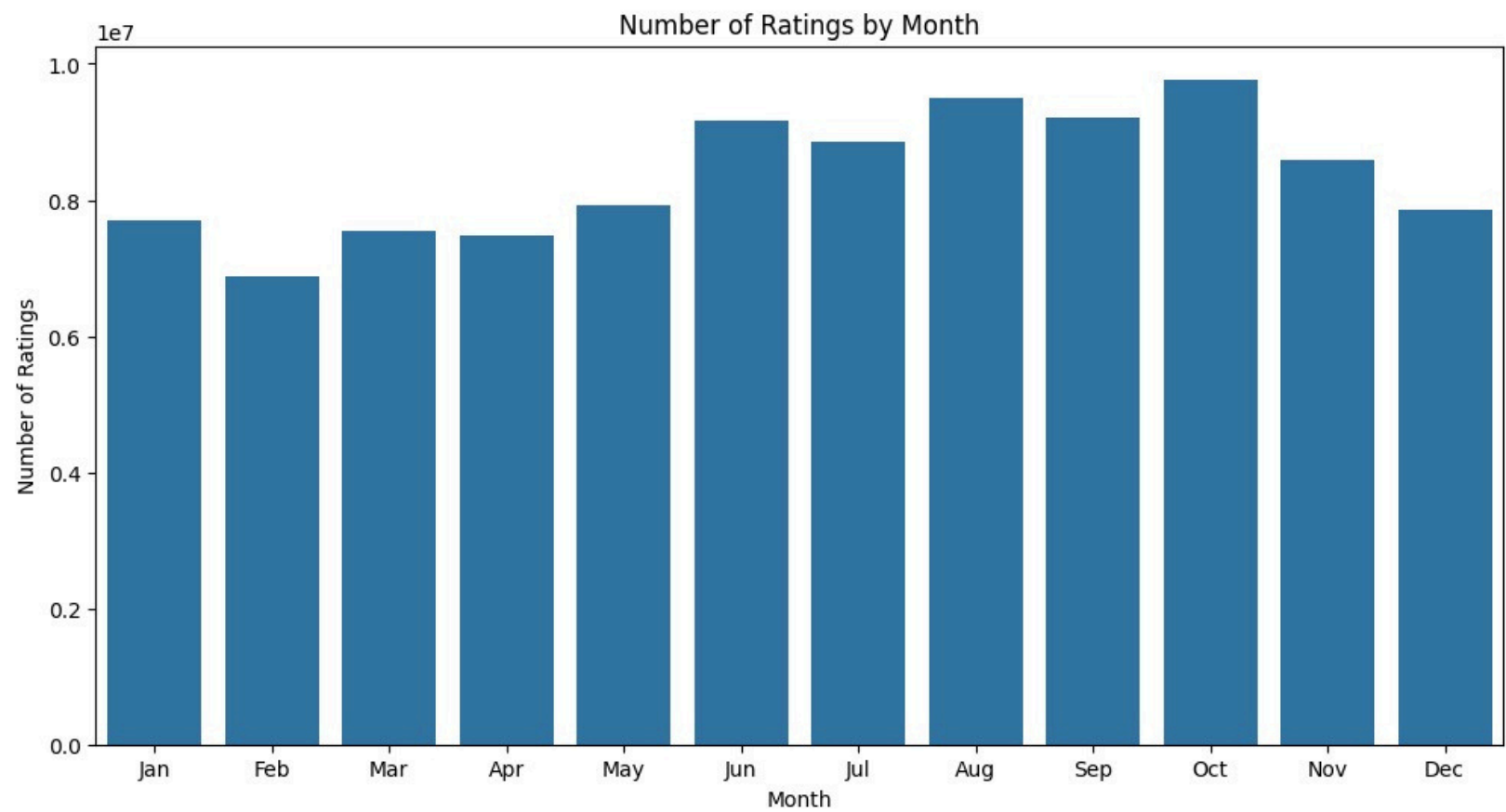
return train_df, test_df
```

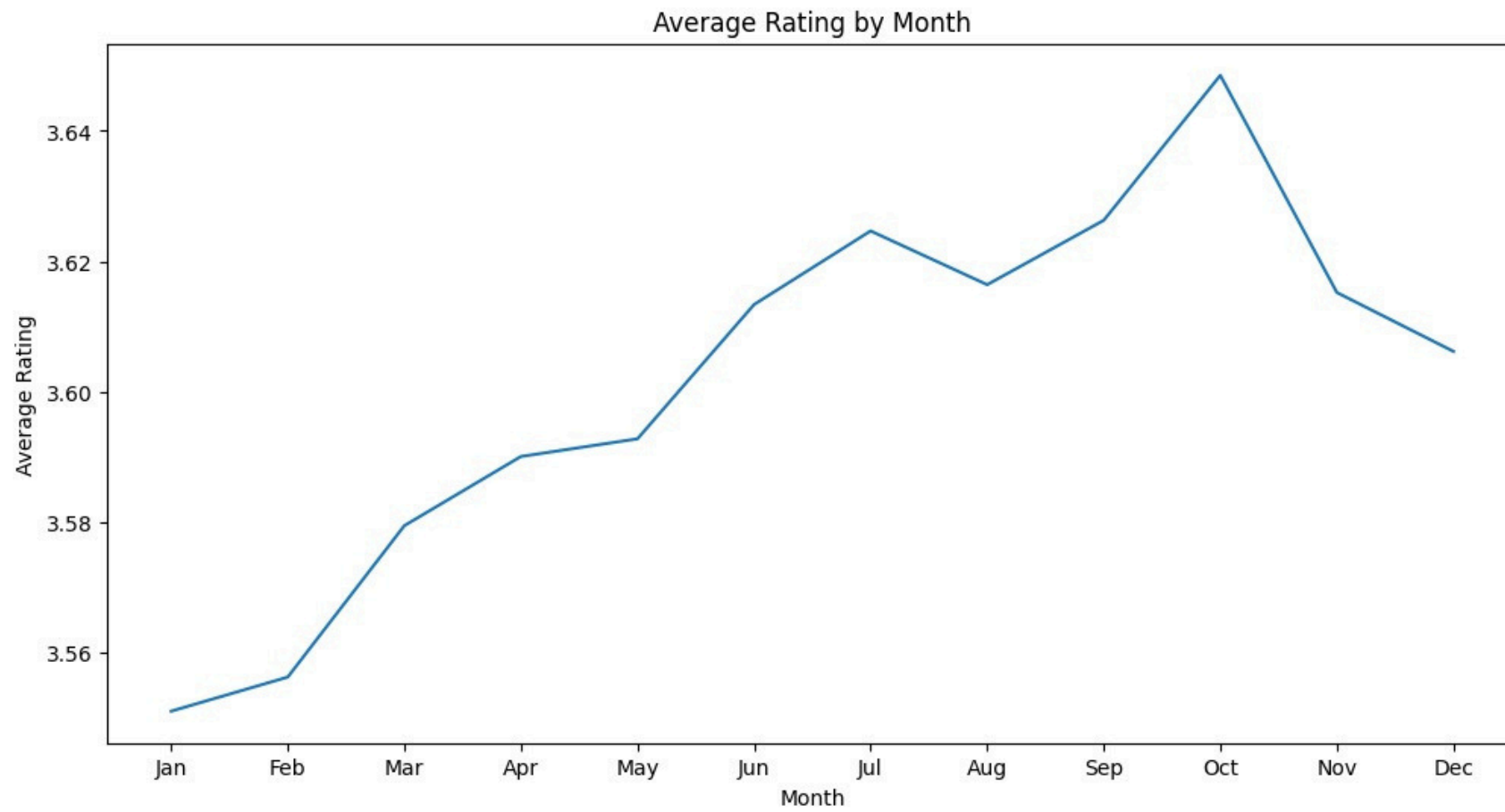
Exploratory Data Analysis

The background of the slide is white with abstract green geometric shapes on the right and bottom-left edges. These shapes are composed of various shades of green, from light lime to dark forest green, arranged in a layered, overlapping fashion. A thin, light gray line runs diagonally across the lower right portion of the slide, intersecting the green shapes.

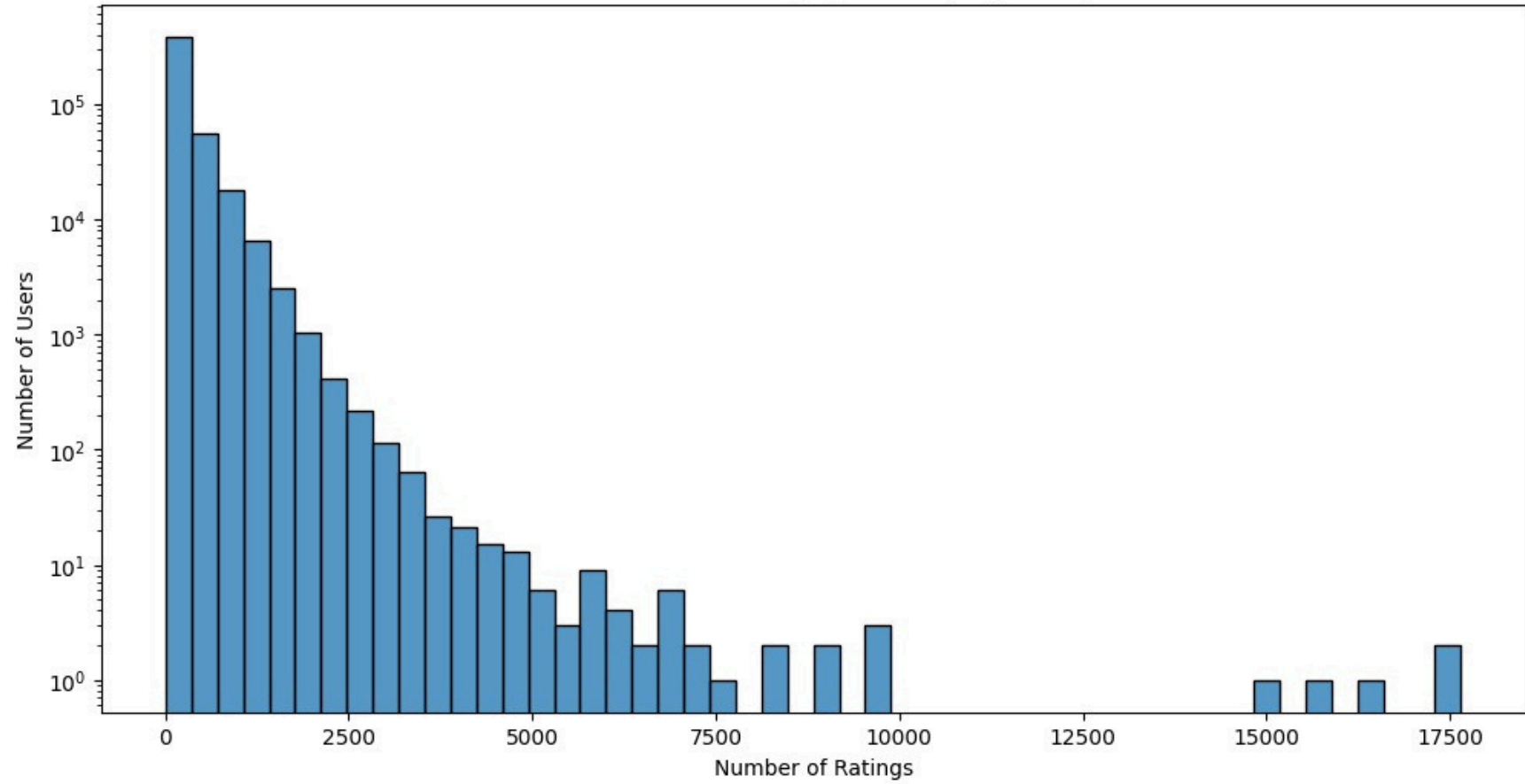


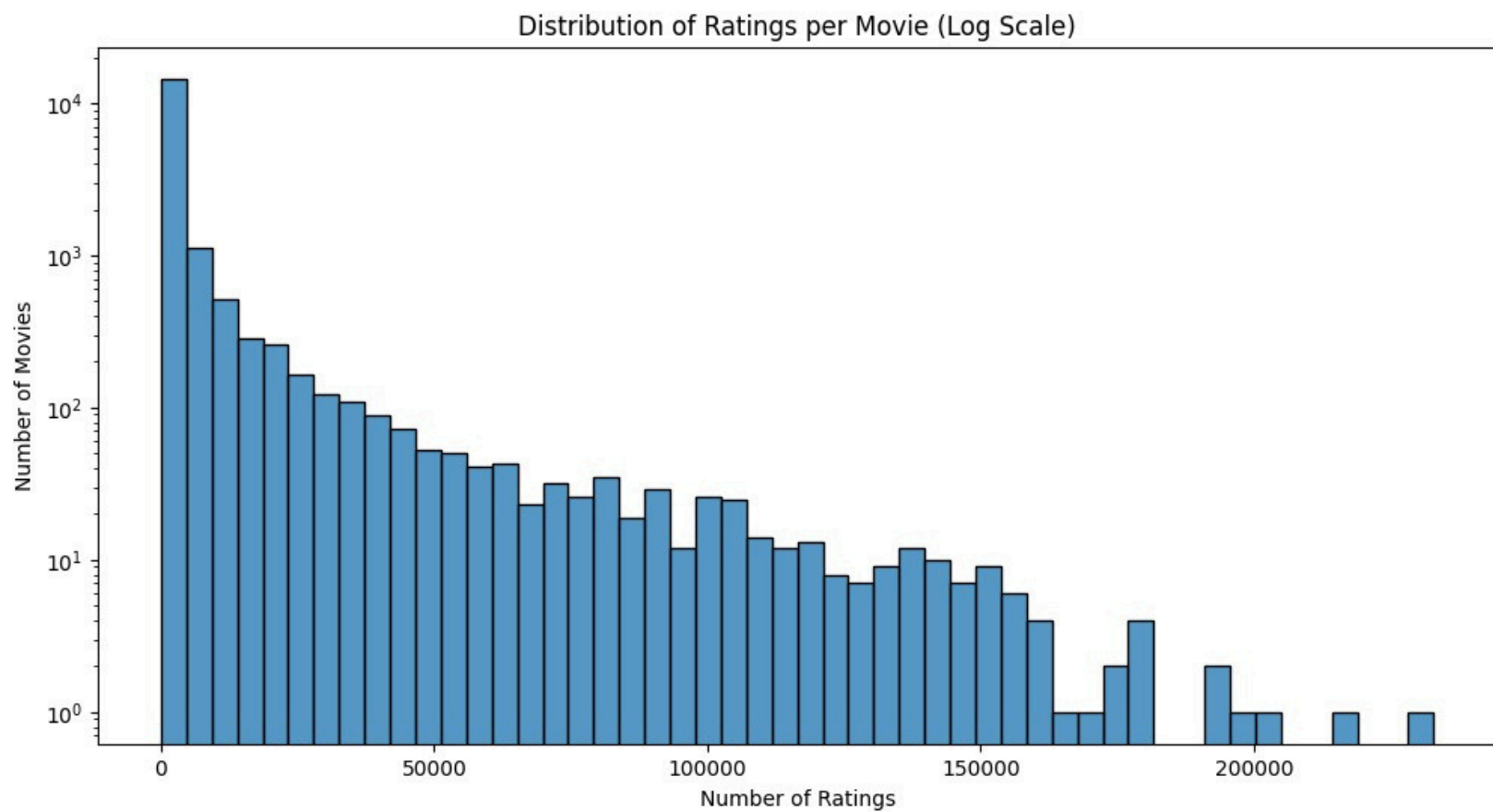


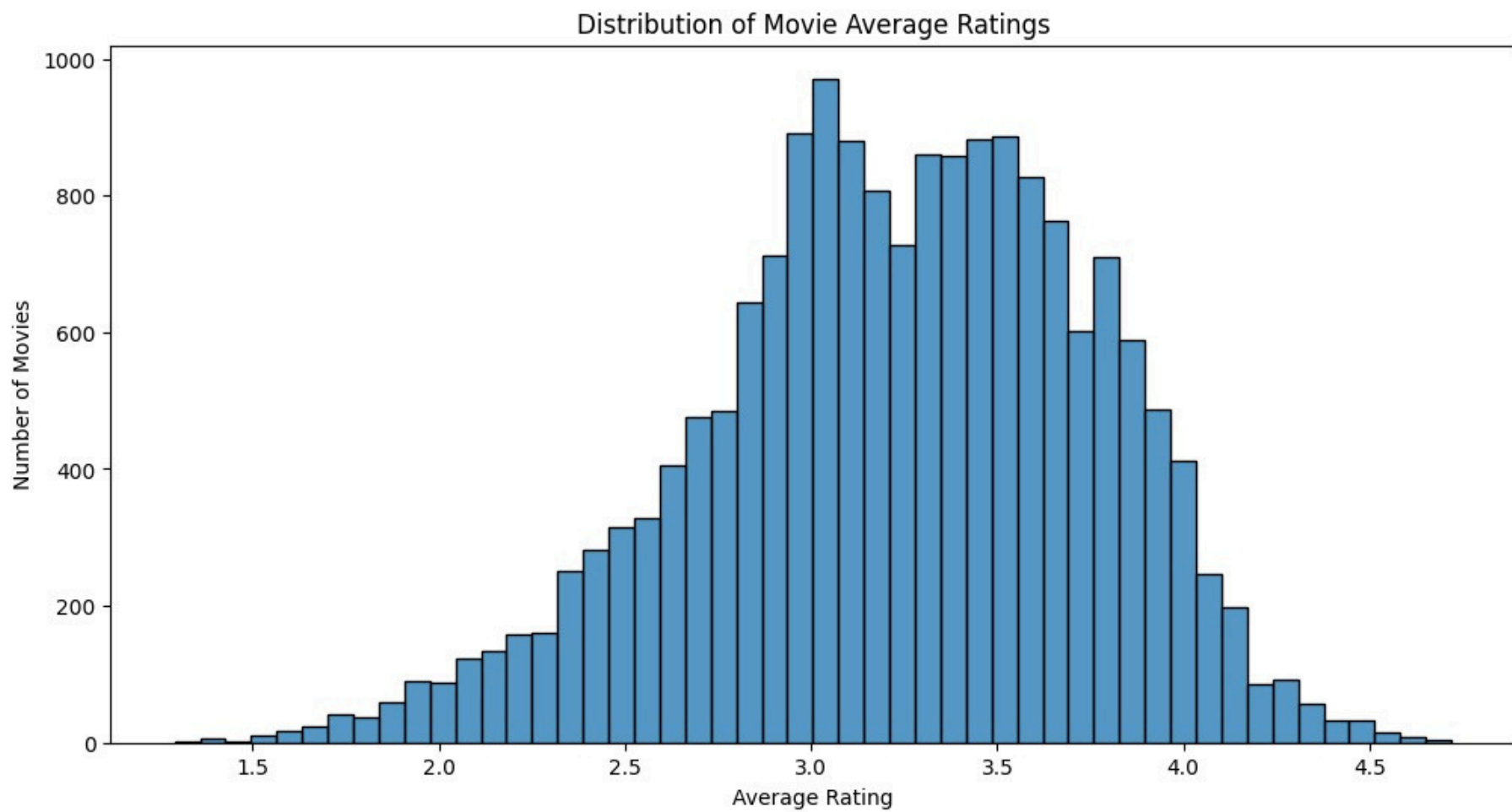




Distribution of Ratings per User (Log Scale)







Matrix Factorization Model

The background of the slide features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green. These shapes are concentrated on the right side and bottom, creating a modern, layered effect. A thin, light gray line also extends diagonally across the lower right portion of the slide.



Matrix Factorization Overview

Core Concept

Decompose user-movie rating matrix R into two lower-dimensional matrices:

$R (n \times m)$

$\approx$

$U (n \times k)$

$\times$

$M (k \times m)$

- n : number of users
- m : number of movies
- k : number of latent factors (50)



Model Parameters

Hyperparameters

n_factors:	50
learning_rate:	0.005
regularization:	0.02
n_epochs:	20
batch_size:	1000000

Model Variables

- User factors matrix (U)
- Movie factors matrix (M)
- User bias vector
- Movie bias vector
- Global bias (mean rating)



Training Process

Training Steps

1. Initialize matrices with small random values
2. For each epoch (20 total):
 - Shuffle training data
 - Process in batches of 1M ratings
 - Update factors using SGD
 - Calculate RMSE
 - Save best model state
 - Reduce learning rate by 10%

Model Implementation

Key Implementation Features

- Efficient memory usage:
 - Float32 data types
 - Sparse matrix operations
 - Garbage collection
- Optimized training:
 - Batched processing
 - Vectorized operations
 - Adaptive learning rate
- Prediction handling:
 - Cold start handling
 - Rating clipping (1-5)
 - Batched predictions

Key function -fit:

Initialization Phase:

- Creates mappings between user/movie IDs and internal indices for efficient storage
- Initializes user and movie factor matrices with small random values (mean=0, std=0.05)
- Sets up bias terms:
 - User bias vector (starts at zeros)
 - Movie bias vector (starts at zeros)
 - Global bias (average of all ratings)

Data Preparation:

Converts the ratings DataFrame into a numpy array with mapped indices

- Structure: [user_index, movie_index, rating]
- Frees memory by deleting original DataFrame
- Uses float32 dtype for memory efficiency

```
for epoch in range(self.n_epochs):
    # Shuffle all ratings
    np.random.shuffle(ratings_array)
    epoch_loss = 0

    # Process in batches
    for batch in tqdm(range(n_batches), desc=f'Epoch {epoch + 1}/{self.n_epochs}'):
        start_idx = batch * self.batch_size
        end_idx = min((batch + 1) * self.batch_size, n_ratings)
        batch_ratings = ratings_array[start_idx:end_idx]

        # Process each rating in the batch
        u_ids = batch_ratings[:, 0].astype(np.int32)
        m_ids = batch_ratings[:, 1].astype(np.int32)
        ratings = batch_ratings[:, 2]

        # Calculate predictions
        predicted = (
            self.global_bias +
            self.user_bias[u_ids] +
            self.movie_bias[m_ids] +
            np.sum(self.user_factors[u_ids] * self.movie_factors[m_ids], axis=1)
        )

        # Calculate errors
        errors = ratings - predicted
        epoch_loss += np.sum(errors ** 2)

        # Update biases
        self.user_bias[u_ids] += self.learning_rate * (errors - self.regularization * self.user_bias[u_ids])
        self.movie_bias[m_ids] += self.learning_rate * (errors - self.regularization * self.movie_bias[m_ids])

        # Update latent factors
        user_factors_grad = (errors[:, np.newaxis] * self.movie_factors[m_ids] -
                             self.regularization * self.user_factors[u_ids])
        movie_factors_grad = (errors[:, np.newaxis] * self.user_factors[u_ids] -
                               self.regularization * self.movie_factors[m_ids])

        # Update factors with vectorized operations
        np.add.at(self.user_factors, u_ids, self.learning_rate * user_factors_grad)
        np.add.at(self.movie_factors, m_ids, self.learning_rate * movie_factors_grad)
```


- **Training Loop:**
- Iterates through n_epochs (20 by default)
- For each epoch:
 - Shuffles all ratings randomly
 - Processes data in batches (1M ratings per batch)
 - Tracks best model state using RMSE
- **Batch Processing:**
- For each batch:
 - Extracts user IDs, movie IDs, and ratings
 - Makes predictions using current model state:

$$\text{prediction} = \text{global_bias} + \text{user_bias} + \text{movie_bias} + (\text{user_factors} * \text{movie_factors}).\text{sum}()$$
 - Calculates prediction errors
 - Updates model parameters using gradient descent

Parameter Updates:

- Updates bias terms:
 - Adjusts user and movie biases based on errors
 - Includes regularization to prevent overfitting
- Updates latent factors:
 - Computes gradients for user and movie factors
 - Uses vectorized operations for efficiency
 - Applies regularization to prevent overfitting
 - Updates using learning rate
- **Model Improvement:**
- Tracks RMSE for each epoch
- Saves best model state when RMSE improves
- Reduces learning rate by 10% after each epoch (learning rate decay)

```
# Calculate RMSE for this epoch
rmse = np.sqrt(epoch_loss / n_ratings)
print(f'Epoch {epoch + 1} RMSE: {rmse:.4f}')

# Save best model
if rmse < best_rmse:
    best_rmse = rmse
    self.best_user_factors = self.user_factors.copy()
    self.best_movie_factors = self.movie_factors.copy()
    self.best_user_bias = self.user_bias.copy()
    self.best_movie_bias = self.movie_bias.copy()

# Reduce learning rate over time
self.learning_rate *= 0.9
```

Fast User-Based Collaborative Filtering



Algorithm Overview

User-Based Collaborative Filtering

Core concept: Predict ratings based on similar users preferences

Key Parameters

- 50 neighbors (k_neighbors)
- 20 minimum ratings
- 50,000 max users

Main Components

- User similarity calculation
- Rating prediction
- Cold start handling



Data Preprocessing

Filtering Steps

- Remove users with < 20 ratings
- Remove movies with few ratings
- Sample users if $> 50,000$
- Create efficient ID mappings

Matrix Creation

- Convert to sparse CSR matrix
- Calculate user means
- Track rating counts
- Store global statistics

Memory Optimizations

- Sparse matrix format saves memory
- Integer-based ID mappings
- Efficient numpy arrays



Similarity Calculation

Batch Processing Algorithm

1. Process users in batches of 100
 - Normalize ratings by user means
 - Calculate cosine similarities
2. For each user:
 - Find top 50 similar users
 - Store user indices and scores

Memory Management

- Batch processing
- Store only top-k neighbors
- Efficient similarity storage

Optimization Techniques

- Vectorized operations
- Sparse matrix usage
- Efficient filtering



Rating Prediction

Prediction Process

1. Get similar users
2. Retrieve their ratings
3. Calculate weighted average
4. Clip to range [1, 5]

Edge Cases

- Cold start: return global mean
- Few ratings: use user mean
- No similar users: use user mean

Batch Prediction Features

- Process predictions in batches of 10,000
- Progress tracking with tqdm
- Memory-efficient processing
- Vectorized operations for speed



Implementation Details

Key Libraries

- numpy: Array operations
- scipy.sparse: CSR matrix
- pandas: Data handling
- sklearn: Cosine similarity

Data Structures

- CSR matrix for ratings
- Dictionary for similarities
- Numpy arrays for means
- ID mapping dictionaries

Performance Optimizations

- Memory efficiency:
 - Sparse matrices
 - Selective storage
 - Batch processing
- Speed optimization:
 - Vectorized operations
 - Efficient filtering
 - Smart data structures


```
class FastUserBasedCF:
    def __init__(self, k_neighbors=50, min_ratings=20, max_users=50000):
        """
        Optimized User-Based Collaborative Filtering

        Args:
            k_neighbors (int): Number of similar users to consider (reduced from 100)
            min_ratings (int): Minimum number of ratings for a user (increased from 5)
            max_users (int): Maximum number of users to consider for similarity calculation
        """
        self.k_neighbors = k_neighbors
        self.min_ratings = min_ratings
        self.max_users = max_users
```

Class Initialization:

- Takes 3 key parameters:
 - k_neighbors: Number of similar users to consider (50)
 - min_ratings: Minimum ratings per user (20)
 - max_users: Maximum users to process (50,000)
- These parameters help balance accuracy vs computational efficiency

Fit Method - Data Preprocessing:

- Filters users and movies:
 - Removes users with too few ratings (< min_ratings)
 - Removes movies with too few ratings
 - If too many users, randomly samples max_users
- Creates efficient mappings between IDs and indices
- Converts ratings to a sparse matrix format (CSR) for memory efficiency

```
print("Creating user and movie mappings...")
# Create mappings
self.user_ids = pd.factorize(filtered_df['user_id'])[1]
self.movie_ids = pd.factorize(filtered_df['movie_id'])[1]

self.user_map = {id_: i for i, id_ in enumerate(self.user_ids)}
self.movie_map = {id_: i for i, id_ in enumerate(self.movie_ids)}

self.n_users = len(self.user_ids)
self.n_movies = len(self.movie_ids)

print("Creating user-movie matrix...")
# Convert to sparse matrix
user_idx = filtered_df['user_id'].map(self.user_map).values
movie_idx = filtered_df['movie_id'].map(self.movie_map).values
ratings = filtered_df['rating'].values

self.rating_matrix = csr_matrix(
    (ratings, (user_idx, movie_idx)),
    shape=(self.n_users, self.n_movies)
)
```

User Statistics Calculation:

- Computes mean rating for each user
- Tracks number of ratings per user
- Stores global mean rating for cold start cases
- Uses sparse matrix operations for efficiency

Similarity Calculation(Key Algorithm Part):

- Processes users in small batches (100 users at a time)

For each batch:

- Normalizes ratings by subtracting user means
- Calculates cosine similarity between batch users and all users

Stores top k similar users for each user

- Optimization techniques:
 - Batch processing to manage memory
 - Uses numpy's efficient array operations
 - Stores only top k neighbors instead of full similarity matrix

```
for i in tqdm(range(0, self.n_users, batch_size)):
    batch_end = min(i + batch_size, self.n_users)
    batch_users = np.arange(i, batch_end)

    # Get normalized ratings for batch
    batch_matrix = self.rating_matrix[batch_users].toarray()
    for j, user in enumerate(batch_users):
        batch_matrix[j] = np.where(
            batch_matrix[j] != 0,
            batch_matrix[j] - self.user_means[user],
            0
        )

    # Calculate similarities
    batch_sim = cosine_similarity(batch_matrix, self.rating_matrix.toarray())

    # Store top k neighbors for each user
    for idx, user in enumerate(batch_users):
        sim_scores = batch_sim[idx]
        # Get top k+1 similar users (including self)
        top_k_idx = np.argpartition(sim_scores, -(self.k_neighbors + 1))[-(self.k_neighbors + 1):]
        # Remove self-similarity
        top_k_idx = top_k_idx[top_k_idx != user][:self.k_neighbors]

        self.similarities[user] = {
            'users': top_k_idx,
            'scores': sim_scores[top_k_idx]
        }
```

□ Prediction Method:

Handles various cases:

- Cold start (new users/movies) → returns global mean
- Users with too few ratings → returns global mean
- Normal case → weighted average of similar users' ratings

□ Prediction steps:

- Get similar users and their similarity scores
- Retrieves their ratings for the target movie
- Calculates weighted average based on similarities
- Clips final prediction to range [1, 5]

```
def predict(self, user_id, movie_id):
    """Predict rating for a user-movie pair"""
    # Handle cold start
    if user_id not in self.user_map or movie_id not in self.movie_map:
        return self.global_mean

    user_idx = self.user_map[user_id]
    movie_idx = self.movie_map[movie_id]

    # If user doesn't have enough ratings, return global mean
    if self.user_rating_counts[user_idx] < self.min_ratings:
        return self.global_mean

    # Get similar users and their ratings
    sim_users = self.similarities[user_idx]['users']
    sim_scores = self.similarities[user_idx]['scores']

    # Get ratings for the movie
    neighbor_ratings = self.rating_matrix[sim_users, movie_idx].toarray().flatten()

    # Filter out users who haven't rated
    mask = neighbor_ratings != 0
    neighbor_ratings = neighbor_ratings[mask]
    sim_scores = sim_scores[mask]

    if len(neighbor_ratings) == 0:
        return self.user_means[user_idx]

    # Calculate weighted average
    weighted_sum = np.sum(neighbor_ratings * sim_scores)
    sim_sum = np.sum(sim_scores)

    if sim_sum == 0:
        return self.user_means[user_idx]

    predicted = weighted_sum / sim_sum
    return np.clip(predicted, 1, 5)
```

□ Batch Prediction:

Processes prediction requests in batches

- Uses tqdm for progress tracking
- Returns numpy array of predictions

```
def predict_pairs(self, pairs_df, batch_size=10000):
    """Predict ratings for multiple pairs"""
    predictions = []

    for start_idx in tqdm(range(0, len(pairs_df), batch_size), desc="Making predictions"):
        batch_df = pairs_df.iloc[start_idx:start_idx + batch_size]
        batch_preds = [self.predict(row['user_id'], row['movie_id'])
                        for _, row in batch_df.iterrows()]
        predictions.extend(batch_preds)

    return np.array(predictions)
```

Results

